FTMO_MT5 · ARCHITECTURE REPORT

An Independent AI Trading Agent for the FTMO_MT5 System

What the system already does, what Sir and the team still do by hand, and exactly where an AI agent fits — built on the code that exists today, not on generic AI theory.

Item	Detail
Prepared	30 September 2026
Basis	Direct inspection of the FTMO_MT5 source tree (~22,000 lines of Python across ftmo_mt5/, mcp_server/, systematic/, broker_hub.py), live FTMO symbol specifications, and results already produced by the research pipeline.
Scope	Paper trading only. Live trading stays disabled; nothing here enables it.
Audience	Sir, the trading team, and whoever builds the agent.

How to read the status labels

Label	Meaning
BUILT	Exists and works today. The agent uses it as-is.
REUSABLE	Code exists but nothing calls it yet. Wiring, not writing.
NEEDS IMPL.	Has to be built. Scope is described in this report.
HUMAN	A named person decides. The agent may recommend, never decide.
HAZARD	Exists today and is unsafe for this purpose. Fix before building on it.

Contents

1 Executive summary
2 The system today
3 What the team does by hand — and what the agent takes over
4 Target architecture
5 The agent's operating rhythm
6 Stage-by-stage design
7 Tool contract
8 Worked FTMO examples
9 Safety architecture
10 Human approval matrix
11 Delivery roadmap
12 What the agent will not solve
A Evidence — file and line references
B Glossary

1 Executive summary

FTMO_MT5 already contains most of what an AI trading agent needs in order to be safe: a research lab that grades strategies honestly, a risk engine that enforces FTMO's daily and overall loss limits, an append-only audit log, and a paper-trading pipeline that cannot reach a real broker. What is missing is the layer that runs this machinery on a schedule, reads the results, explains them, and hands decisions to a person at the right moments. That layer is the agent.

> **The design in one sentence**
> The agent should not be a trader that happens to have safeguards. It should be an analyst and planner with no ability to trade, sitting on top of deterministic code that does.

Five findings from the code

#	Finding	Why it matters	Status
1	**An AI agent already runs inside the hub.** /agents is a general-purpose Claude Code runner: shell access, file edits, acceptEdits by default, bypassPermissions selectable from the page — in the same process as the order routes.	It must not become the trading agent. With a shell it can call the hub's own order endpoint, bypassing the import boundary that makes the MCP server safe.	HAZARD
2	**The live loop is disconnected from research.** main.py trades one hardcoded EMA-crossover rule. Nothing the Lab validates ever reaches it.	The agent's plans have one destination: the paper pipeline. The live loop stays out of scope.	NEEDS IMPL.
3	**An ML validation layer is written but dormant.** Meta-labeler, drift detection, CSCV, Deflated Sharpe and a promotion gate exist; nothing imports them. The meta-label gate passes every signal when no score is supplied.	Ready-made parts for the agent's validation step. Wiring, not writing.	REUSABLE
4	**Promoted rules are mostly untradable.** Of the five rules research promoted, four — VZ, IEF, TLT, GIS — are not on FTMO's symbol list. Only NKE is.	The pipeline has no tradability check. It is the first concrete job for both the agent and the code.	NEEDS IMPL.
5	**There is no validated edge yet.** 0 ROBUST, 7 MODERATE after calibration fixes — out of 555 screens, which is not clearly better than chance.	The agent's first duty is honest reporting, not trading. The grading rubric already enforces this.	BUILT

Recommendation

1. **Phase 0 — make it safe.** Lock /agents down (tool allowlist, remove bypassPermissions from the page, validate the working directory) and replace the three health checks that always report healthy.
2. **Run the agent as a separate process** on the Claude Agent SDK, with allowed_tools restricted to MCP tools: no shell, no file edits, a dollar budget per run.
3. **Give it tools that read and propose — never execute.** Extend the existing ftmo-lab MCP server with research, paper-account and reporting tools.
4. **Keep every decision with money attached in deterministic code:** grading (verdict.validate), limits and sizing (RiskAgent), fills (PaperBroker), audit (EventStore).
5. **Put a person at five gates:** promoting a rule, approving a cycle plan (until Phase 3), clearing a HALT, changing risk limits, and anything touching live trading.

2 The system today

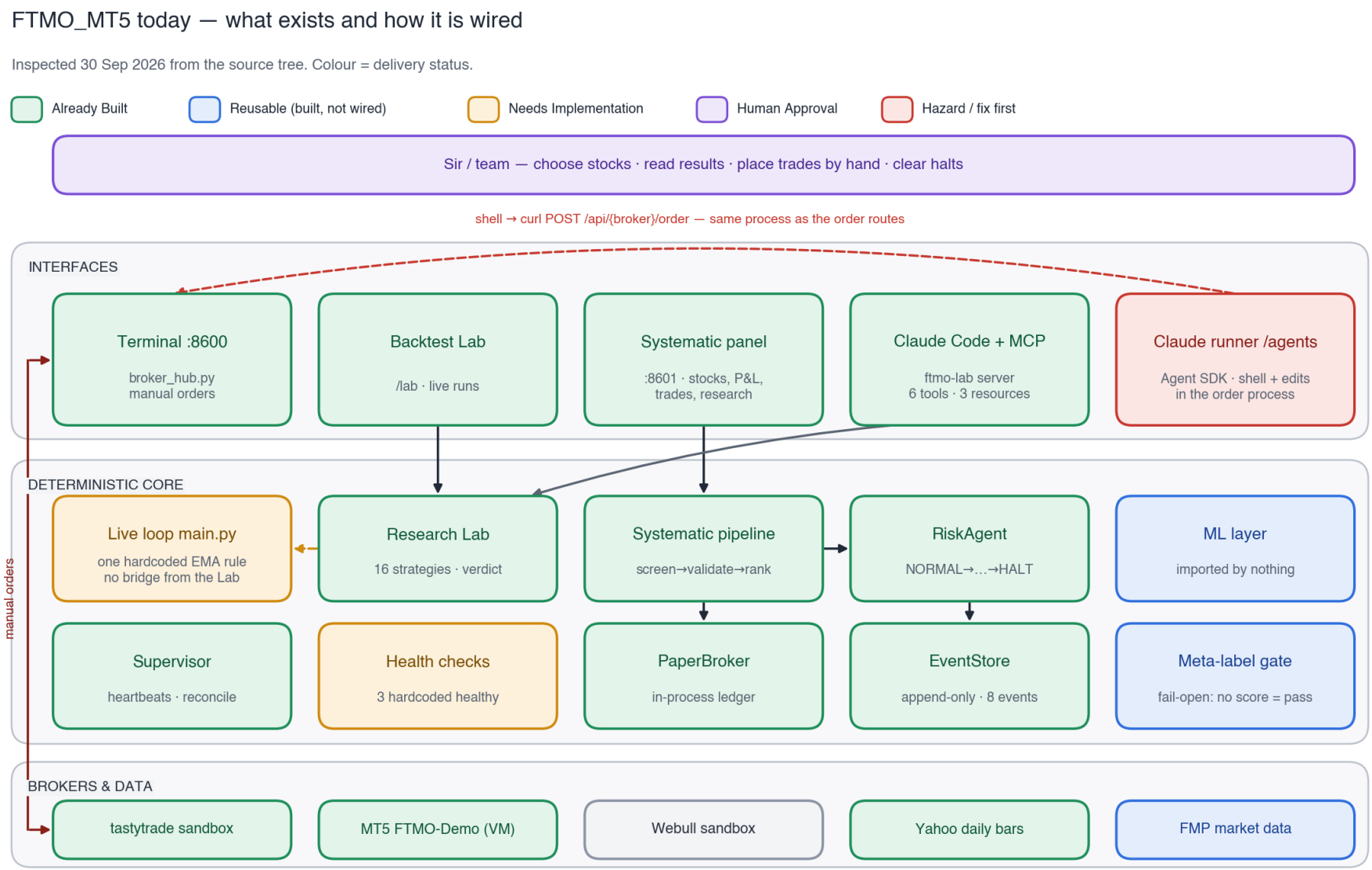


Figure 1 — Components as they exist on 30 Sep 2026. The red dashed arc is the hazard in Finding 1; the amber dashed arrow is the missing Lab → live-loop bridge.

Every component below was located in the source tree and checked for whether anything actually calls it. "Built" means it runs; "Reusable" means the code exists but is not wired into any path.

Component	Where	What it does today	Status	Role for the agent
Research Lab	ftmo_mt5/lab/	Backtests, metrics, robustness grading: ROBUST / MODERATE / FRAGILE / FAILED	BUILT	Research engine it calls
Strategy library	ftmo_mt5/signals/	16 registered rules; parameters typed int / float / bool	BUILT	The menu it selects from — it never writes rules
ftmo-lab MCP server	mcp_server/	6 tools, 3 reference resources, results index of ~3,500 runs	BUILT	Primary tool surface
Systematic pipeline	systematic/	Watchlist → screen → validate → rank → paper cycle	BUILT	Executes the approved plan
RiskAgent + rule engine	agents/risk_agent.py engine/rule_engine.py	NORMAL → CAUTION → RISK_OFF → HALT against FTMO daily and overall loss	BUILT	Final say on every order
PaperBroker	systematic/paper_broker.py	In-process ledger; fills at the close with slippage and commission	BUILT	Where approved trades land
EventStore	store/event_store.py	Append-only audit; 8 event types	BUILT	Source of truth for monitoring and reports
Systematic panel	systematic/webui.py (:8601)	Stocks, P&L, trade history, research, run controls	BUILT	Where approvals get clicked
Supervisor + reconciliation	supervisor/, execution/reconciliation.py	Heartbeats, kill switch, position reconciliation	BUILT	Monitoring inputs
ML layer	ftmo_mt5/ml/	Meta-labeler, drift (PSI / ADWIN), CSCV, Deflated Sharpe, promotion gate	REUSABLE	Extra validation and drift checks
Market data (FMP)	MCP connector	Quotes, company profiles, earnings and economic calendars	REUSABLE	Pre-market event scan
Live loop	ftmo_mt5/main.py	Signal → risk → execution with one EMA rule; has never completed a trade cycle	NEEDS IMPL.	Outside the agent's path until Phase 4
Health checks	observability/health.py	Three checks hardcoded to healthy=True	NEEDS IMPL.	Must be real before the agent trusts them
Claude runner /agents	broker_hub.py:1316–1476	Free-form Claude Code tasks with shell and edits, in the hub process	HAZARD	Developer tool only — lock down, never promote

3 What the team does by hand — and what the agent takes over

Read across a row: today's manual work, the agent's job, what stays in deterministic code, delivery status, and where a person still decides.

Stage	Today — Sir / team	The agent	Stays deterministic	Status	Human gate
1 Stock screening	Picks tickers by judgement and types them into My Stocks. A one-off FTMO top-25 ranking was produced on request on 30 Sep.	Rebuilds the FTMO-tradable universe weekly from FTMO's symbol list; ranks by liquidity, ATR fit and overnight-gap risk; flags delistings.	Ticker validation (universe.py); live-data check on add	NEEDS IMPL.	Approve watchlist changes
2 Market analysis	Reads news, charts and quotes informally. No written brief.	Pre-market brief: overnight moves, earnings and macro calendar, gap risk per open position, data freshness.	Nothing — analysis only	NEEDS IMPL.	None — informational
3 Strategy selection	Chooses which rule to try on which stock.	Selects only from the 16 registered rules and the current ranking. Never invents or edits a rule.	Strategy registry and rule code	BUILT	None — selecting is not deploying
4 Backtesting	Runs the Lab page or a Claude session ad hoc.	Runs research on a schedule through MCP; checks the results index before re-running; every run is logged.	Lab engine, cost model, windows	BUILT	None
5 Validation	Reads the verdict and decides whether it is good enough.	Explains why each rule graded as it did, and states how many tests were run so a lucky pass is visible.	verdict.validate — the grade itself	BUILT	Promote a rule (≥ MODERATE)
6 Trade planning	Clicks Paper cycle after the close, then reads what happened.	Dry-runs the cycle and writes a plan with a reason per trade; checks FTMO tradability, earnings blackout and exposure.	Cycle logic; fresh-signal rule	NEEDS IMPL.	Approve the plan (Phase 2)
7 Risk checks	Watches equity and trusts RiskAgent.	Explains risk state and remaining daily-loss room in dollars; warns before a threshold, not after.	RiskAgent, rule engine, sizing	BUILT	Change risk limits
8 Execution	Clicks Paper cycle; places manual orders in the terminal.	**Nothing.** The agent holds no order tool.	PaperBroker (later: sandbox adapter)	BUILT	Any live mode
9 Monitoring	Opens dashboards; runs curl to confirm paper mode.	Tails EventStore; detects HALT, stale data, broker-mode drift and reconciliation breaks; raises an incident.	Supervisor, kill switch, HALT	NEEDS IMPL.	Clear a HALT
10 Reporting	Writes manager updates by hand (a manager-report skill exists).	Daily P&L, weekly research memo, incident reports, and a proposed-vs-executed diff.	EventStore as the source of every number	NEEDS IMPL.	Review before sending
Ops Environment	Starts the VM, hub and port forwarder; checks mode with curl.	Verifies paper mode and data freshness before any work; refuses to continue if either fails.	start.sh; LIVE_TRADING_ENABLED = False	NEEDS IMPL.	None

4 Target architecture

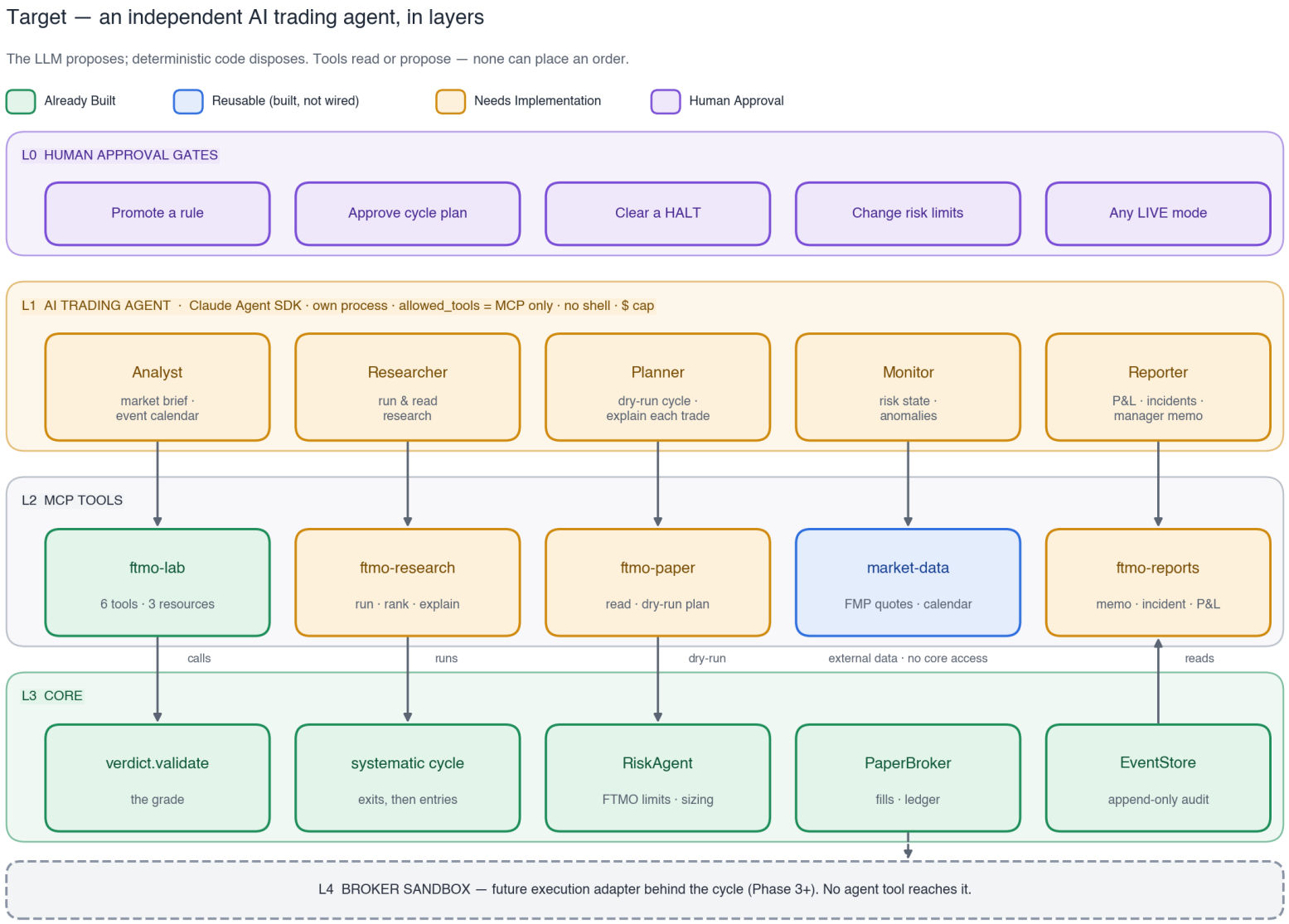


Figure 2 — Five layers. The agent's authority is its tool list: every tool reads or proposes. Execution happens only in L3, after an L0 gate.

Design principles

- **The LLM proposes; deterministic code disposes.** Grades, limits, sizing and fills are computed by code that already exists and is tested.
- **Authority is the tool list, not the prompt.** A prompt can be ignored or manipulated; a missing tool cannot be called.
- **Every number comes from a tool.** The model explains figures; it never produces them. Reports are assembled from tool outputs, with prose around them.
- **Structural safety over behavioural safety.** The same idea already protects the codebase: mcp_server/ and systematic/ cannot import execution code, and a test enforces it.
- **People decide at irreversible or policy-changing moments** — and nowhere else, so approvals stay meaningful instead of becoming a rubber stamp.

Agent process specification

Setting	Value	Why
Runtime	Claude Agent SDK (Python), its own process, started by a scheduler	Isolated from broker_hub and its order routes
allowed_tools	mcp__ftmo-lab__*, mcp__ftmo-research__*, mcp__ftmo-paper__*, mcp__ftmo-reports__*, market-data read tools	The tool list is the permission model
Denied	Bash, Write, Edit, WebFetch, and any broker tool	Removes every path to an order or a code change
permission_mode	default — never acceptEdits or bypassPermissions	No silent writes
max_budget_usd / max_turns	Per run, e.g. $1–2 and a turn cap (the hub already uses a $2 cap)	A loop cannot burn money
Models	Claude Sonnet 5 for daily briefs and plans; Claude Opus 5.5 for the weekly research memo; Claude Haiku 4.5 for monitoring summaries	Match cost to the judgement needed
Working directory	Fixed in code, never taken from a request	The current /agents accepts any cwd from the page

Why not reuse /agents?

It was built as a developer convenience, and its own comment calls it a free-form Claude Code runner. It loads project settings, can edit files, can use the shell, and defaults to acceptEdits. Turning it into a trading agent would mean removing almost everything it is for. Build the trading agent separately; keep /agents as a locked-down developer tool.

5 The agent's operating rhythm

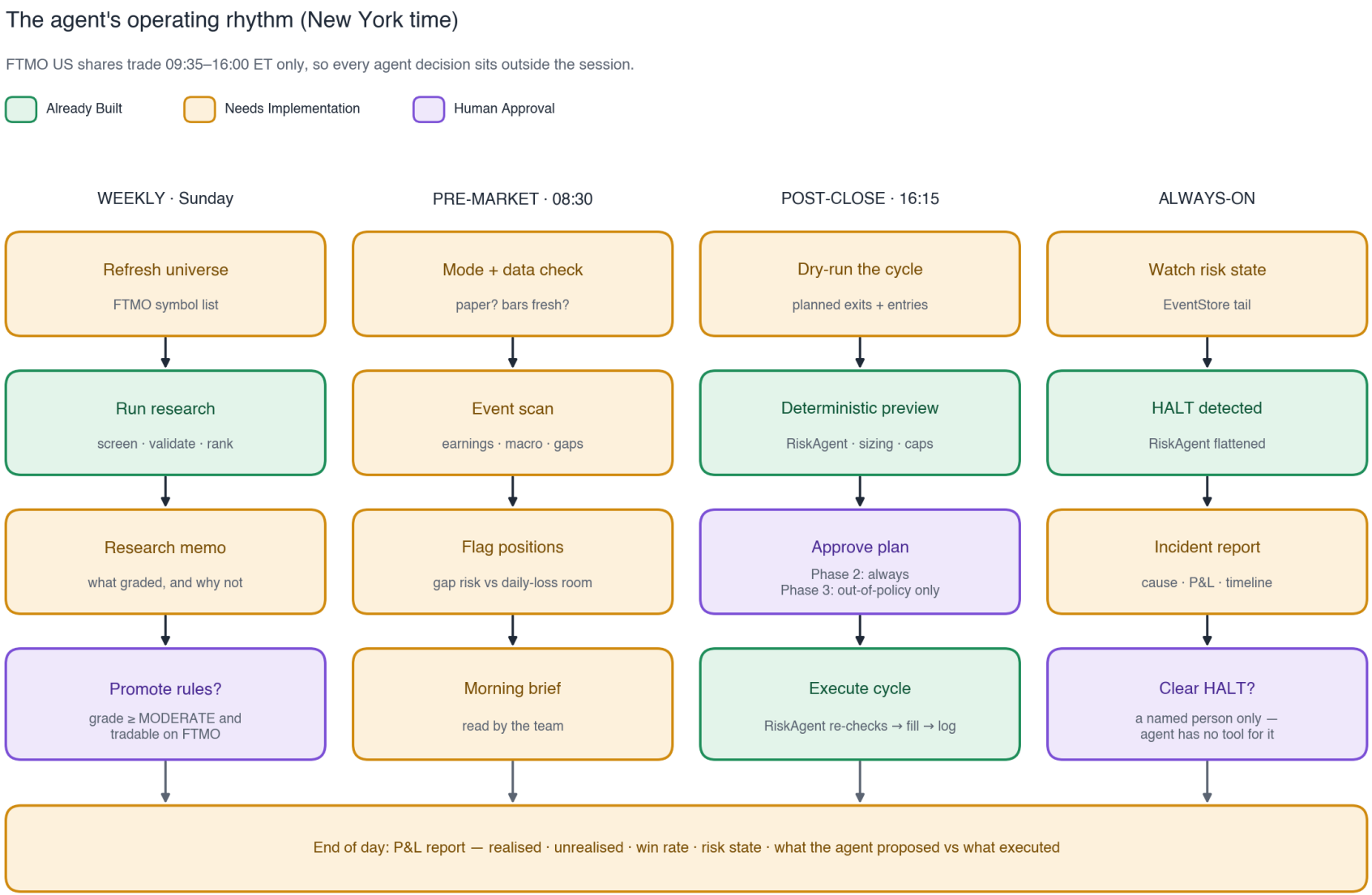


Figure 3 — Four loops. FTMO US equity CFDs trade 16:35–23:00 GMT+3 (09:35–16:00 New York), so every agent decision falls outside the session.

Schedule

When (NY)	Job	Tools used	Output	Gate
Sunday	Refresh the FTMO universe, run research, write the memo	universe_ftmo, research_run, research_ranking, lab_lookup	Research memo with grades, reasons and test count	PERSON
08:30 daily	Mode and data check; event scan; gap flags	risk_status, paper_account, market calendar and quotes	Morning brief	INFO ONLY
16:15 daily	Dry-run the cycle; explain each planned trade	paper_plan, risk_status, research_ranking	Cycle plan	PERSON
Continuous	Watch risk state and events; draft incidents	events_tail, risk_status	Alerts and incident reports	PERSON
End of day	P&L and proposed-vs-executed diff	paper_trades, paper_account, events_tail	Daily report	INFO ONLY

Gate: PERSON means someone acts before anything changes. INFO ONLY means the output changes nothing by itself.

Why 16:15 and not 16:00: the pipeline ignores the day's bar until 16:15 New York time so it never trades on a close that has not been published. The agent inherits that rule rather than re-deciding it.

6 Stage-by-stage design

One card per stage. "Stays in code" is the part the agent may read or trigger but never replace.

1	Stock screening NEEDS IMPL.
Exists today	Watchlist validation checks every ticker against live data before saving (fixed 27 Sep to accept exchange suffixes such as RELIANCE.NS). An FTMO top-25 list was measured once on 30 Sep (systematic/watchlists/ftmo-top25.txt); the scoring script was not added to the repo.
Agent adds	Weekly: read FTMO's symbol list, measure all US-session names over one identical window, rank by liquidity, ATR fit and gap safety, and propose additions and removals with the evidence.
Stays in code	Ticker parsing, data validation, the scoring formula.
Human gate	A person approves watchlist changes.

2	Market analysis NEEDS IMPL.
Exists today	Nothing written. The FMP connector already returns quotes, profiles and calendars in Claude sessions.
Agent adds	08:30 brief: overnight moves on held names, earnings within two sessions, macro releases, gap risk against remaining daily-loss room, and whether today's bars are fresh.
Stays in code	Price data and the completed-bar rule.
Human gate	None — the brief changes nothing by itself.

3	Strategy selection BUILT
Exists today	16 registered rules with typed parameters; research picks the best three per stock for full validation.
Agent adds	Explains which rules were tried and why the winners won. Never proposes a rule outside the registry and never tunes parameters live.
Stays in code	The registry, rule code, parameter types.
Human gate	None — selection is not deployment.

Stage-by-stage design — stages 4 to 6

4	Backtesting BUILT
Exists today	Lab engine and six MCP tools with canonical defaults; results index of ~3,500 runs.
Agent adds	Runs research on schedule; calls lab_lookup first so repeated work is skipped; reports cost assumptions next to results.
Stays in code	Engine, cost model, windows, canonical settings.
Human gate	None.

5	Validation BUILT
Exists today	verdict.validate: 70/30 out-of-sample split, cost ladder, top-trade removal, five walk-forward folds, parameter sensitivity. The ML layer adds CSCV and Deflated Sharpe but is not wired.
Agent adds	States the number of tests behind every pass, so a result from 555 screens reads differently from one from 5. Wires the Deflated Sharpe check in as a second opinion.
Stays in code	The grade. The agent can explain it, never raise it.
Human gate	A person promotes a rule; the floor is MODERATE and FAILED can never be promoted.

6	Trade planning NEEDS IMPL.
Exists today	The cycle computes exits then entries, refuses to join trades older than one bar, and sizes equal-weight slots.
Agent adds	A dry-run tool returns the planned orders without placing them. The agent adds a reason per trade and three checks the pipeline lacks: FTMO tradability, earnings blackout, and exposure against daily-loss room.
Stays in code	Cycle logic, sizing, fresh-signal rule.
Human gate	A person approves the plan (Phase 2); only out-of-policy plans in Phase 3.

Stage-by-stage design — stages 7 to 10

7	Risk checks BUILT
Exists today	RiskAgent state machine; on a $100k 2-Step account CAUTION at $3,000 daily loss, RISK_OFF at $4,000, HALT at $4,500 — ahead of FTMO's $5,000 limit.
Agent adds	Translates state and headroom into plain language and warns in the morning brief when open gap exposure could cross a threshold overnight.
Stays in code	Every limit, every size multiplier, every HALT.
Human gate	A person changes limits.

8	Execution BUILT
Exists today	PaperBroker fills at the completed close with slippage and commission; every order passes RiskAgent first.
Agent adds	Nothing. Execution is triggered by the scheduler or the approval button, never by an agent tool call.
Stays in code	All of it.
Human gate	Live mode needs a separate sign-off outside this system.

9	Monitoring NEEDS IMPL.
Exists today	Supervisor heartbeats and kill switch; EventStore records every signal, risk decision, fill and HALT.
Agent adds	Reads the event log continuously; detects HALT, stale data, broker-mode drift and reconciliation mismatches; drafts an incident report with a timeline.
Stays in code	HALT itself, and its refusal to clear automatically.
Human gate	Only a named person clears a HALT.

10	Reporting NEEDS IMPL.
Exists today	Reports per research run and per cycle as Markdown; a manager-report skill; the panel's P&L view.
Agent adds	Daily P&L, weekly research memo, incidents, and a proposed-vs-executed diff that proves the agent never acted outside its plan.
Stays in code	Numbers come from the ledger and event log.
Human gate	A person reviews anything sent outside the team.

7 Tool contract

The complete list of what the agent can call. Read tools return data; Compute tools run research; Propose tools return a plan without acting; the single Write tool can only write into a reports folder.

Tool	Server	Kind	Status	Notes
lab_list_strategies	ftmo-lab	Read	BUILT	The 16-rule menu
lab_run_backtest	ftmo-lab	Compute	BUILT	Canonical defaults enforced by the server
lab_run_matrix	ftmo-lab	Compute	BUILT	All rules × timeframes
lab_run_batch	ftmo-lab	Compute	BUILT	Many specs in one call
lab_validate_strategy	ftmo-lab	Compute	BUILT	Returns the grade
lab_lookup	ftmo-lab	Read	BUILT	~3,500 indexed runs; call before re-running
research_run	ftmo-research	Compute	NEEDS IMPL.	Wraps python -m systematic research; weekly budget
research_ranking	ftmo-research	Read	NEEDS IMPL.	ranking.json with reasons and test counts
universe_ftmo	ftmo-research	Read	NEEDS IMPL.	FTMO symbol list and tradability check
paper_account / paper_trades	ftmo-paper	Read	NEEDS IMPL.	Ledger and P&L (logic exists in webui._pnl)
paper_plan	ftmo-paper	Propose	NEEDS IMPL.	Dry-run cycle: planned orders, none placed
risk_status	ftmo-paper	Read	NEEDS IMPL.	State plus headroom in dollars
events_tail	ftmo-paper	Read	NEEDS IMPL.	EventStore reader since a timestamp
quotes, calendars, news	FMP connector	Read	REUSABLE	News text is untrusted data, never instructions
report_write	ftmo-reports	Write	NEEDS IMPL.	The only write — into reports/ only

Deliberately not exposed

- **Order routes** — /api/{broker}/order and /close. No broker gateway is reachable from any agent tool.
- **Running the cycle** — triggered by the scheduler or the approval button, never by the agent.
- **reset-halt** — requires --by NAME --confirm from a person, by design (hard rule 4).
- **Settings and risk limits** — the agent may suggest a change in a report; it cannot write one.
- **Watchlist writes** — propose-only; a person saves the change in the panel.
- **Shell and file edits** — the single biggest difference from today's /agents runner.

8 Worked FTMO examples

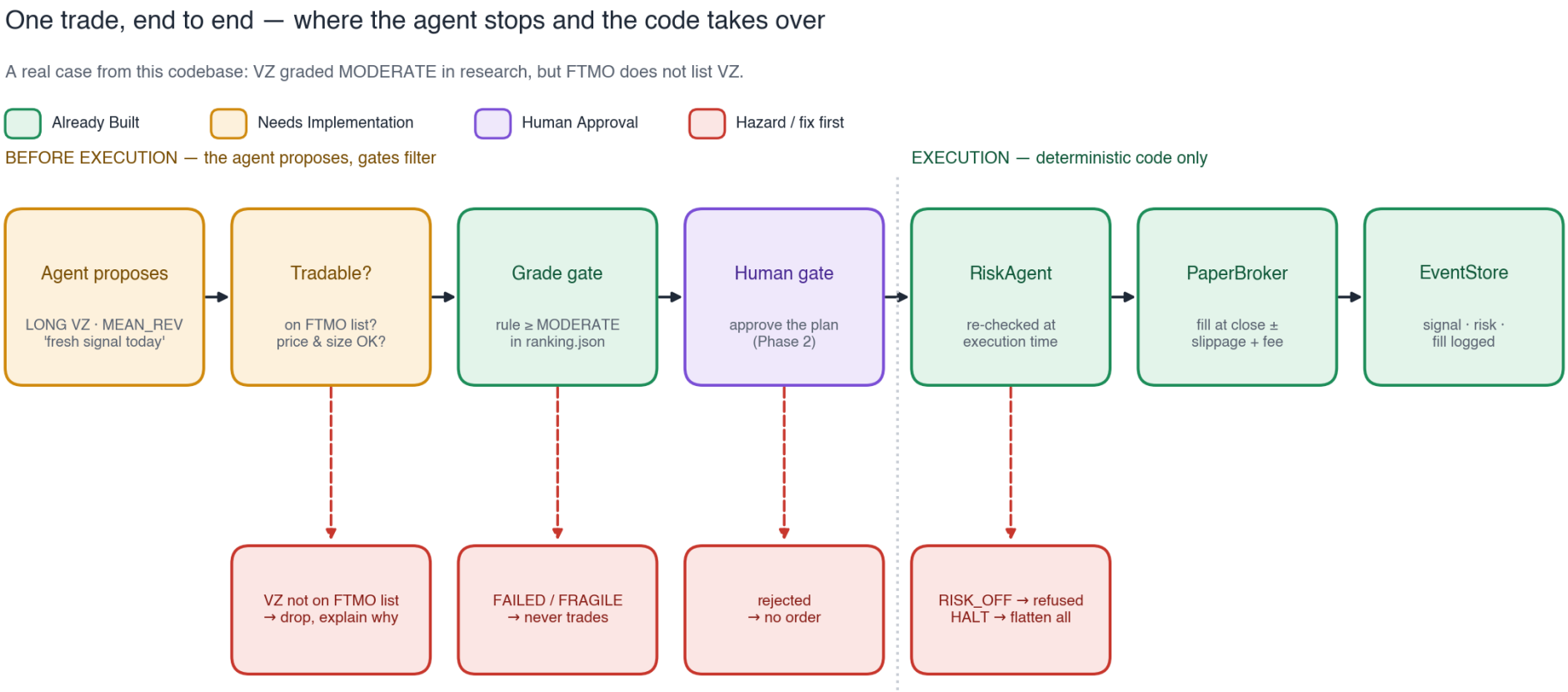


Figure 4 — One proposal through every gate, using a real case from this codebase: VZ graded MODERATE, but FTMO does not list VZ.

Every figure below comes from this codebase's own runs or from FTMO's published specifications. None is illustrative.

A. A MODERATE rule the account cannot trade

The 10-year research run on 26 Sep promoted five rules at the default MODERATE floor. Cross-checked against FTMO's US-session equity list on 30 Sep:

Symbol	Rule	Grade	On FTMO?
VZ	MEAN_REVERSION	MODERATE	**No** — not listed
IEF	DONCHIAN_BREAKOUT	MODERATE	**No** — an ETF; FTMO lists single stocks
TLT	MEAN_REVERSION	MODERATE	**No** — an ETF
GIS	MEAN_REVERSION	MODERATE	**No** — not listed
NKE	MEAN_REVERSION	MODERATE	**Yes** — Nike, Spot CFD

What the agent does

Drops four of five candidates before planning, and says why in the memo. It also recommends a code change: put the tradability check into the pipeline itself, so the grade and the market list can never disagree again. The agent finds the gap; the fix belongs in deterministic code.

B. The daily-loss ladder on a $100,000 2-Step account

State	Daily loss	Agent action	Code action
NORMAL	under $3,000	Morning brief states headroom in dollars	NORMAL — full size
CAUTION	$3,000	Alerts the team; lists open exposure and its gap risk	CAUTION — new orders at half size
RISK_OFF	$4,000	Drafts an incident note; proposes no entries	RISK_OFF — entries refused, exits allowed
HALT	$4,500	Writes the incident report; cannot clear the HALT	HALT — flatten everything, stay halted
FTMO BREACH	$5,000	—	FTMO's own limit — the account fails

Thresholds from risk_agent.py (lines 6–8); dollar values assume the $100,000 initial capital in config/default.yaml. Colour marks severity.

C. A real HALT, and what the agent writes

A 250-day replay of the five promoted rules halted on an FTMO daily-loss breach: equity fell to $99,414 against a floor of $99,693. Realised profit had lifted the day's starting balance to about $104,700, so the open float counted against the day's allowance.

Position	Side	Closed P&L	Note
NKE	LONG 175	−$2,451.75	Largest contributor
VZ	SHORT 471	−$2,190.15	Short into a rally
GIS	LONG 249	−$749.49	
IEF	SHORT 105	+$101.85	

The agent's incident report — and where it stops

Cause: four positions opened across several weeks carried a combined float of about −$5,300; FTMO measures daily loss on equity, so the float counted in full.

Finding: the rules hold through drawdowns deep enough to fail a challenge. Equal-weight slots of 20% of equity are too large for them.

Recommendation: cut the slot size or cap total open float, then re-run the replay.

It cannot clear the HALT. A named person reviews the report and runs reset-halt --by NAME --confirm.

D. Overnight gaps on a market that only trades by day

FTMO's US shares trade 09:35–16:00 New York. A gap overnight cannot be traded out of and lands directly on the daily-loss limit. Measured over the year to 29 Sep 2026:

Symbol	95th-pct gap	On a $20,000 slot	Share of the $3,000 CAUTION trigger
AMD	6.14%	$1,228	41% from one position in one night
AVGO	3.65%	$730	24%
AAPL	1.43%	$286	10%
MCD	1.27%	$254	8%

What the agent does

In the 08:30 brief it checks the earnings calendar and adds up worst-case gap exposure across open positions. If a name reports within one session, or the combined 95th-percentile gap would cross CAUTION, it proposes no new entry in that name. The cycle still decides; the agent only makes the risk visible before the market opens.

E. Auditing the research itself

The Lab's default commission, $2.50 per unit per side, is a futures figure. Applied per share it consumed $5,755 of $8,226 gross profit (70%) in a real-data replay. Correcting it raised the FRAGILE count from 3 to 20; widening the window to 2,500 bars then produced the project's first MODERATE grades.

> **What the agent does**
> Every research memo prints cost per round trip beside average profit per trade, and trade count beside the 30-trade MODERATE gate. Either comparison would have caught this calibration error on day one.

9 Safety architecture

Each risk is specific to this codebase, and each control is either already built or scoped in the roadmap.

Risk	How it would happen here	Control	Status
Agent places an order	Shell → curl the hub's order route (possible today via /agents)	Separate process; MCP-only allowlist; no broker tool; import boundary already tested	NEEDS IMPL.
Prompt injection	A headline or filing says "buy X now"	Market text is data; no tool executes; plans still pass every deterministic gate	NEEDS IMPL.
Invented numbers	The model states a Sharpe or P&L it never computed	Reports assembled from tool outputs; the model writes prose around them	NEEDS IMPL.
P-hacking	The agent re-runs research until something passes	Weekly research budget; the results index logs every run; memos state tests run	REUSABLE
Runaway cost	A loop burns API spend	max_budget_usd and max_turns per run	BUILT
Stale data	Trading on a bar still forming	Completed-bar rule at 16:15 ET; freshness check in the brief	BUILT
Broker-mode drift	Credentials switched to a live account	Mode assertion before every run; LIVE_TRADING_ENABLED = False	BUILT
Silent failure	Health checks always report green	Replace the three hardcoded checks (Phase 0)	NEEDS IMPL.
HALT override	The agent "fixes" a halt to resume	No reset tool exists; reset needs --by NAME --confirm	BUILT

The one to fix first

Today, a person on this machine can open /agents, switch to bypassPermissions, and ask for anything the shell can do — including POST /api/tasty/order. The hub binds to 127.0.0.1, which keeps it off the network, but not away from its own order routes. Phase 0 closes this before any agent work starts.

10 Human approval matrix

Decision	The agent may	A person must	Why
Change the watchlist	Propose with evidence	Approve and save	Changes the universe traded
Run research	Run on schedule, within budget	—	Reversible and logged
Promote a rule	Recommend, with grade and reasons	Approve	Puts capital behind a rule
Daily cycle plan	Draft it, with a reason per trade	Approve (Phase 2); exceptions only (Phase 3)	First decision on the money path
Execute an order	Never	Never directly — the cycle executes	Keeps proposal and execution apart
Clear a HALT	Write the incident report	Review, then clear with --confirm	Hard rule 4: never auto-recover
Change risk limits or sizing	Suggest in a report	Approve and edit config	Policy, not operations
Enable live trading	Never	Separate sign-off outside this system	Out of scope

Phase 3 replaces per-plan approval with a policy envelope — maximum positions, gross exposure, per-name caps, earnings blackout — agreed once and enforced in code. Plans inside the envelope run; anything outside it waits for a person.

11 Delivery roadmap



Figure 5 — Indicative timing. Each phase has an exit test; the next phase does not start until it passes.

Phases and exit tests

Phase	Deliverables	Exit test	Effort
0 Make it safe	Lock /agents: allowed_tools, remove bypassPermissions from the page, fixed working directory. Replace the three hardcoded health checks.	/agents cannot run Bash or reach an order route; each health check fails when its dependency is down.	S · 2–4 days
1 Read-only analyst	ftmo-research, ftmo-paper (read) and ftmo-reports MCP servers. Agent process with MCP-only tools. Morning brief and research memo.	Ten consecutive briefs; every number traceable to a tool call; zero writes outside reports/.	M · 1.5–2 weeks
2 Proposer + human gate	paper_plan dry-run tool. Approval screen in the :8601 panel. Tradability and earnings checks moved into the pipeline.	Twenty approved cycles; proposed and executed orders match exactly; no agent-initiated order.	M · 2–3 weeks
3 Autonomous paper	Policy envelope enforced in code; auto-run inside it; alerts outside it. Four-week forward test.	Forward results compared with validation; no daily-loss breach; incidents handled without manual log reading.	L · 2 weeks + 4-week test
4 Live	Out of scope for this document.	Separate sign-off, capital limits, and a new review of every control above.	—

Effort assumes one engineer part-time with Claude Code. Phase 3's length is fixed by the forward test, not by build effort: daily rules fire one or two times a week each, so four weeks is the minimum that says anything.

12 What the agent will not solve

- **It does not create an edge.** The standing result is 0 ROBUST. An agent makes research faster, more honest and better documented; it cannot make a losing rule profitable.
- **Position sizing is the open problem.** The replay HALT in Example C came from sizing, not rule choice. That is engineering and policy work for people.
- **A short forward test is weak evidence.** Four weeks of daily rules is tens of trades — enough to catch broken plumbing, not enough to prove profitability.
- **Demo still enforces FTMO's rules.** Simulated money does not mean breaches are free on a paid challenge account.

Appendix A — Evidence

Every finding in this report can be checked at these locations in the FTMO_MT5 source tree (inspected 30 Sep 2026).

Claim	Location	Check
/agents runs the Claude Agent SDK in the hub	broker_hub.py:1316–1476	Read the section header comment
Default permission mode is acceptEdits	broker_hub.py:1357	AgentRunRequest.permissionMode
Working directory comes from the request	broker_hub.py:1352, 1372	cwd field, passed straight through
No tool allowlist on the runner	broker_hub.py:1365–1382	_agent_options sets no allowed_tools
Hub binds to loopback only	broker_hub.py:1559	host="127.0.0.1"
Live loop uses one EMA crossover rule	ftmo_mt5/main.py:305–307	EMACrossoverStrategy()
ML layer is imported by nothing	ftmo_mt5/ml/	grep for 'ftmo_mt5.ml' outside ml/
Meta-label gate is fail-open	agents/signal_agent.py:122	if meta_label_prob is not None
RiskAgent thresholds	agents/risk_agent.py:6–8	2-Step: 3% / 4% / 4.5%
Six MCP tools	mcp_server/server.py:164–406	lab_* functions
Paper pipeline cannot go live	systematic/__init__.py:20	LIVE_TRADING_ENABLED = False
Import boundary is enforced by a test	tests/test_systematic.py:38	Imports the package in a clean process
Four of five promoted rules untradable	systematic/state-10y/ranking.json	Compare with ftmo.com/en/symbols

Appendix B — Glossary

Term	Meaning
MDL	Maximum Daily Loss — FTMO's limit on loss within one day, measured on equity including open positions. $5,000 on a $100,000 2-Step account.
ML (loss)	Maximum Loss — FTMO's overall limit; equity may never fall below $90,000 on a $100,000 2-Step account.
ROBUST / MODERATE / FRAGILE / FAILED	The Lab's grades. Only MODERATE and above may be promoted; FAILED never.
ATR	Average True Range — the typical size of a day's move, here as a percentage of price.
MCP	Model Context Protocol — how the agent is given tools. Its tool list is its permission model.
Claude Agent SDK	The library that runs Claude as an agent with a defined tool list, budget and permission mode.
Dry run	Computing what the cycle would do without placing any order.
Policy envelope	Limits agreed in advance (positions, exposure, blackouts) inside which a plan may run without a per-plan approval.